



Briefing Notes

This activity will take place in DB G.30 and will take 3 hours. You should attend the session shown on your [personal timetable](#). You will be working individually or in groups of 2 and the maximum total number of students is 100.

Health and Safety

The approved Risk Assessment for this activity, reference number: RA_5815, is available on the University LabCup system: <https://mmu.uk.labcup.net/>.



The class will start with a Health and Safety briefing. Students who miss this will not be allowed to take part until they are briefed - please make sure you arrive in good time.

Attendance Record

You may be required to sign an attendance sheet in the laboratory. Please note that this is for the unit teaching team's local records only and does not replace the requirement for [attendance registration](#) for participating students.

Aim

The aim of this experiment is to:

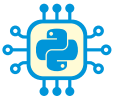
- Introduce data processing structures and methods.
- Introduce file input and output for data storage.

Background Reading

- Thonny, Python IDE for beginners: <https://thonny.org/>
- LearnPython.org interactive Python tutorial: <https://www.learnpython.org/>

About this resource

Document last updated:	September 2026	<i>Note that printed or emailed copies of this document may be out of date. Always check Moodle for the latest version.</i>
Next scheduled review date:	August 2027	
Comments and feedback on this document should be sent to the author(s):		Tim Collins



1 Sequences

1.1 Data Types, Objects and Methods

In the last session, we used several types of variable, e.g. int, float, bool etc. We will be expanding that list in this session.

In general, a variable refers to a data *object*. Each data object has a data *type* that defines what kind of data is stored in the object, how much memory is needed, and what *methods* are associated with the object. In the following sections, we will look at more data types that are commonly used in embedded programming for representing sequences of multiple values.

We often want to work with not just one or two numbers but, instead, with sequences of variables (e.g. representing samples of an audio file or pixels in an image). There are several ways of representing sequences in Python including *lists*, *tuples*, *dictionaries*, *sets* and *bytearrays*.



Video Introduction to Sequences and Lists
https://mmutube.mmu.ac.uk/id/1_wd0e5kjg

1.2 Lists

A list is defined by entering a sequence of values, separated by commas inside square brackets. We used one in the first session to define the sequence of values used by a 'for' loop. A variable can be assigned to represent a list, rather than just a single value, for example:

Shell

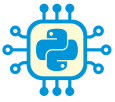
```
z = [3,1,4,1,5,9]
```

assigns the specified list of six numbers to the variable, z.

In effect, this creates six separate variables. To read or write to a single element in the list, square brackets are used to specify the index to that element. Indices start counting at 0 and finish at the array length minus one (5 in this case). For example:

```
1 z = [3,1,4,1,5,9]
2 print(z[0])           # Prints element 0 = 3
3 print(z[5])           # Prints element 5 = 9
4 z[1] = 99             # Changes the value of element 1
5 print(z)              # Prints the whole list: [3,99,4,1,5,9]
```

Important Point: List indices start counting at zero; `z[0]` is the first item in the list above, `z[1]` is the second, and so on. This is the same convention as used in C, C++, Java, etc. but not by MATLAB which counts from one instead of zero.



List members can be of any data type and do not have to all be the same type. E.g. A list containing an int, a string, a float, a bool and a complex member can be created:

Shell

```
mylist = [13, 'hello', 3.14159, True, 1+3j]
```

A list member can also be another list itself (or any other kind of sequence). This allows multi-dimensional lists to be created. E.g. To create a Python representation of the two-dimensional grid below, try the following code:

	0	1	2
0	a	b	c
1	d	e	f

```
1 x = [['a', 'b', 'c'], ['d', 'e', 'f']]
2 print(f"x[0] = {x[0]}")
3 print(f"x[1] = {x[1]}")
4 print(f"x[1][0] = {x[1][0]}")
```

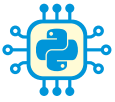
In this example, `x` is a list containing two lists. To access an element within one of those lists, two indices are needed: the first selects the list and the second the element within that list. So, the last line of the example above selects list index 1, `['d', 'e', 'f']`, and element zero from that list, which is `'d'`. Referring to the original grid above, the first index gives the row number and the second gives the column.

Exercise: Write a script that encodes the following table in a two-dimensional list: parts

Resistor	100
Capacitor	1e-6
Inductor	0.2
Transistor	BC108

1. Print the value of the third line of the list. This should display: `['Inductor', 0.2]`
2. Print the value of the capacitor (line 1, item 1). Should display: `1e-06`
3. Add the line: `parts[3][1] = '2N0000'` and print the entire list to check the effect is what you expected.

You can find a [sample solution here](#) if you get stuck or want to check your answer.



1.3 Tuples

A *tuple* behaves very much like a list in most respects: it is an ordered sequence of values, each of which can be accessed by an index number. The difference is that tuples are what is known as *immutable*. This means that, once created, you cannot change members of a tuple and you cannot add or remove members. Tuples are declared by listing the members in regular 'round' brackets rather than square ones. E.g.

```
1 x = [2, 4, 6, 8]
2 print(type(x))           # Should print: <class 'list'>
3 y = (2, 4, 6, 8)
4 print(type(y))           # Should print: <class 'tuple'>
5
6 print(x[2])
7 print(y[2])              # Both should print: 6
8
9 x[1] = 9                  # This is fine but would NOT work with y (tuples are read-only)
```

What are Tuples for?

Tuples can be a confusing concept at first - why would you ever want to use a tuple instead of a list? There are several reasons:

- There are many cases where you do not want members of a sequence to be changed (e.g. due to a programming error). Tuples guarantee this.
- Tuples require less memory to store them - this can be important on resource-constrained devices such as microcontrollers.
- Using tuples can result in faster execution times, there is a lot less behind-the-scenes overhead in managing a constant, unchangeable sequence compared with a list.

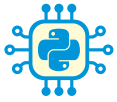
In general, the advice is to use a tuple whenever possible and to only use a list when you need to.

1.4 Strings

A *string* is a special type of sequence made up of text characters. All the other types of sequence can contain arbitrary data types as their members but strings only contain text. We have been using strings already as arguments to the `print()` function. Strings behave a lot like tuples in that once created, you cannot alter them although there are a lot of functions and methods that can create new, modified versions.

String variables are defined using quotation marks (either single or double) and can be manipulated in similar ways to other variables. For example:

```
1 s1 = "abc"
2 s2 = 'def'
3 print(s1+s2)             # Concatenates two strings; prints 'abcdef'
4 print(s2[1])             # Access element at index 1 of the string, s2: 'e'
```



The `input()` function returns a string which is why we had to use an `int()` function as well, earlier, to turn it into an integer. To input a line of text as a string simply use:

Shell

```
txt = input()
```

Substrings

As well as accessing single letters, we often want to split a string at a specified index to form smaller substrings. For example,

```
1 s = "Embedded Systems"
2 print(s[6:12])           # Prints letters at indices, 6,7,8,9,10,11 'ed Sys'
3 print(s[:5])             # Prints the first 5 letters: 'Embed'
4 print(s[9:])             # Prints from index 9 to the end: 'Systems'
```

NB. The same notation can be used with lists and tuples as well to access sub-sets of a sequence.

1.5 Dictionaries



Video Guide: Dictionaries another example use case.

https://mmutube.mmu.ac.uk/id/1_d7022svh

A dictionary is another sequence which, like a list, can be created and then modified later by changing elements, adding new elements, etc. Unlike a list, however, a dictionary does not store the elements in any particular order - instead, they are organised in *key-value* pairs.

For example, imagine a bill of materials for a project that has the line:

Component	Value	Quantity
Resistor	120	5

This could be represented using a list, e.g.

Shell

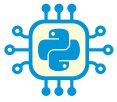
```
comp = ['Resistor', 120, 5]
```

However, this requires the coder to remember what each element in the list means. Does this list represent 120 5Ω resistors or 5 120Ω resistors?

Try the follow code to see how we could use a dictionary instead:

```
1 comp = {'Component': 'Resistor', 'Value': 120, 'Quantity': 5}
2 print(comp)
3 print(comp['Value'])           # Note, keys are used instead of integer indices
4 comp['Tolerance'] = 0.05      # You can add extra key-value items later
5 print(comp)
```

Note that the dictionary is defined using 'curly' brackets and elements are accessed using the keys rather than by an index.



With the exception of strings, all sequences can be made up of any data type including other sequences. For example, a list of dictionaries describing a bill of materials might be:

```
1 c1 = {'Component': 'Resistor', 'Value': 1000, 'Quantity': 2}
2 c2 = {'Component': 'Capacitor', 'Value': 1e-6, 'Quantity': 1}
3 bom = [c1, c2]
4 print(bom)
```

Exercise: Write a script to encode the data in the table below as a list of dictionaries, each with the keys: 'Code', 'Title' and 'Semester'.

Code	Title	Semester
6E5Z0000	Analogue Electronics	1
6E5Z0003	Digital Electronics	1
6E5Z0001	Control Engineering	2

[Sample solution](#) if you get stuck or want to check your answer.

1.6 Sets



Video Guide: Sets in Python.

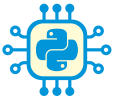
https://mmutube.mmu.ac.uk/id/1_1osts5qf

Sets behave a little differently to the other sequences in that the members are not ordered and not indexed, i.e. you cannot access individual entries in a set using an index or key. Also, members of a set are all unique - if you add the same value twice to a set, only one will get stored. Sets are declared using 'curly' brackets but are distinct from dictionaries because no keys are used. E.g.

```
1 x = {2, 3, 5, 3, 9, 2}
2 print(x)           # Should display {9, 2, 3, 5} - no duplicates, order is random
```

There are several special operators for use on sets, for example '&' can be used to find the intersection of two sets and '|' to find the union. E.g.

```
1 x = {'a', 'c', 'd', 'c'}
2 y = {'b', 'a', 'd'}
3 print(x)           # {'c', 'a', 'd'}
4 print(y)           # {'b', 'd', 'a'}
5 print(x|y)         # {'c', 'a', 'd', 'b'}
6 print(x&y)         # {'a', 'd'}
```



1.7 Bytearrays



Video Guide: Bytearrays in Python.

https://mmutube.mmu.ac.uk/id/1_e8qwadlf

Bytearrays behave much like lists except that the elements of a bytearray are all single bytes. Bytearrays occupy a contiguous block of memory and are often used for streaming blocks of data in or out of a processor. There are several ways to create a byte array, e.g.

```
1 a = bytearray(3)           # a = [0, 0, 0] (all single bytes)
2 b = bytearray([2, 4, 6, 8]) # b = [2, 4, 6, 8] (all single bytes)
```

Bytearrays are mainly used for low-level data transfers and are commonly required in microcontroller programming.

2 Working with Sequences

2.1 Casting Sequences

Sometimes, we need to convert from one sequence to another. Just like other data types, there are casting functions that achieve this when possible. E.g.

```
1 a = [1, 2, 4, 8]           # Creates a list:      [1, 2, 4, 8]
2 b = tuple(a)               # Creates a tuple:    (1, 2, 4, 8)
3 c = bytearray(b)           # Creates a bytearray: [1, 2, 4, 8]
4 d = set(c)                 # Creates a set:      {1, 2, 4, 8}
5 e = list(c)                # Creates a list again.
```

2.2 List Methods

Python data types often have specific *methods* associated with them. Methods are functions that act specifically on the object that they are connected with. For lists, a simple example is the `append()` method that appends an additional item on the end of an existing list. For example:

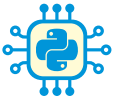
```
1 lst = [1, 2, 4, 8]
2 print(lst)           # Prints: "[1, 2, 4, 8]"
3 lst.append(16)        # Uses the append method of this list to add an extra item
4 print(lst)           # Prints: "[1, 2, 4, 8, 16]"
```

Note that, in order to use a method, the object must already have been initialised.

There are several other `list` methods available.

See https://www.w3schools.com/python/python_lists_methods.asp for more details.

Exercise: Write a script that takes the list: `p = [3, 11, 7, 13, 1, 5, 2]`, re-arranges the elements of `p` in ascending order and then prints the resulting list. Refer to the `list` methods documented at the link above to find a method for re-arranging the list elements.



2.3 String Methods

Strings share many of the same methods and functions that lists have, but also have additional text-specific methods too. For example, there are several methods that can manipulate the case of a string:

```
1 txt = "play it again Sam"
2 print(txt.upper())      # Prints all in capitals
3 print(txt.lower())      # Prints all in lower case
4 print(txt.capitalize()) # Capitalises first letter only
```

For details of more string methods, see: https://www.w3schools.com/python/python_strings_methods.asp

2.4 The 'in' Operator

A very common requirement is to check whether an element is in a sequence or not. The 'in' operator achieves this for any kind of sequence including lists, sets, dicts and strings:

```
1 primes = [2, 3, 5, 7, 11]
2 print(3 in primes)      # True
3 print(4 in primes)      # False
4 print(6 not in primes)   # True
```

Note that 'not in' returns True if the element is absent from the sequence. The same operators work for other sequences including strings:

```
1 print('a' in 'cake')    # True
2 print('i' in 'team')     # False
```

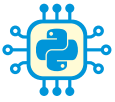
For dictionaries, the 'in' operator tests whether the element is one of the keys of the dictionary:

```
1 s = {'Length': 73, 'Width': 19, 'Height': 20}
2
3 print('Length' in s)     # True - 'Length' is one of the keys
4 print(19 in s)           # False - 19 is one of the values but not a key
```

Exercise: Write a script that defines a function: `containsLetterA(word)` that returns True if the argument, `word`, contains the letter 'A', or False otherwise. Make sure you can handle upper or lower case characters

Test your function for the following cases.

1. `containsLetterA('Miranda')` # should return True
2. `containsLetterA('Ariel')` # should return True
3. `containsLetterA('Umbriel')` # should return False



2.5 Iterating over Sequences

If you want to perform the same actions on each member of a list, you can access the elements one by one using a `for` loop:

```
1 z = [3,1,4,1,5,9]
2 for i in range(6):
3     print(z[i])
```

or, more compactly:

```
1 z = [3,1,4,1,5,9]
2 for x in z:           # x takes the values of z[0], z[1], z[2], etc one-by-one
3     print(x)
```

Both have the same result, printing the numbers in the list, `z`, one-by-one. Note that, when used with 'for', the 'in' operator returns each value, one-by-one, that 'in' is True for.

Exercise: Write a script that takes a list, `z = [4,6,2,8]`, calculates the product of all its elements and prints the result. Use a for-loop to step through the list members multiplying a variable, `prod`, by each one. NB. `prod` will need initialising to one at the start.
(The result should be 384)

2.5.1 Useful functions used for iterations

The first iteration example in used `range(6)` to iterate over the indices of the list. This assumes that the length of the list is known to be 6. For a more general solution, the list length can be calculated using the function: `len()`. E.g.

```
1 z = [3,1,4,1,5,9]
2 for i in range(len(z)): # len(z) works out the list length, 6 in this case
3     print(z[i])
```

You will often want to iterate over the members of more than one sequence. For example, given two lists, `a = [1, 4, 6, 2]` and `b = [3, 2, 4, 8]`, if you need to create a third list, `c` whose members are each the sum of the corresponding elements in `a` and `b` (i.e. `c = [1+3, 4+2, 6+4, 8+2] = [4, 6, 10, 10]`).

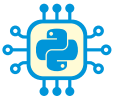
A simple way of doing this is:

```
1 a = [1, 4, 6, 2]
2 b = [3, 2, 4, 8]
3 c = []           # Initialise c as an empty list
4 for i in range(len(a)):
5     c.append(a[i] + b[i]) # Add the i-th members of a and b and append to the list
```

Alternatively, there are two functions used in iterations that can help with this task.

`enumerate()`

The function, `enumerate()`, allows you to iterate through a sequence but on each iteration it returns two values, the index to the current sequence member and the value of that member. The index can then be used to look up values in other sequences:



```
1 a = [1, 4, 6, 2]
2 b = [3, 2, 4, 8]
3 c = []
4 for i,x in enumerate(a): # x will be the members of a: 1, 4, 6, 2
5     c.append(x + b[i])    # i will be the indices: 0, 1, 2, 3
```

zip()

The function `zip()` takes two or more sequences as input parameters and, on the n -th iteration, returns a tuple whose members are the n -th members of the sequence. In this example, `zip(a, b)` returns tuples (1,3), (4,2), (6,4) and (2,8). So, the same task as above can be accomplished by:

```
1 a = [1, 4, 6, 2]
2 b = [3, 2, 4, 8]
3 c = []
4 for z in zip(a, b): # z will iterate through (1,3), (4,2), (6,2) and (2,8) in turn
5     c.append(sum(z)) # sum() returns the sum of all members of a sequence
```

There is no particular advantage of any of these methods for this simple example but, for more complex tasks, these methods can greatly simplify the code required.

Exercise: In an experiment, a list of five voltages is measured and then the experiment repeated two more times to reduce errors. The result is three lists of five voltages:

V1 = [1.96, 4.02, 6.09, 8.12, 9.91]

V2 = [2.03, 3.87, 5.79, 7.98, 9.69]

V3 = [1.98, 3.78, 6.11, 7.93, 10.01] With reference to the `zip()` example above, write a script that calculates a new list, `V_av`, each member of which is the average of the corresponding items in V1, V2 and V3.

2.6 List Comprehension

A concise and powerful way of creating new lists based on the values of existing ones is known as *List Comprehension*. It can be used in several forms, the simplest is in the form:

```
new_list = [new_item for member in existing_sequence]
```

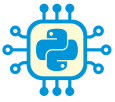
When executed, the variable `member` iterates over all the members of `existing_sequence`; each value is used as part of the calculation of `new_item` and added to the new list. The result is a list but `existing_sequence` can be any iterable sequence including lists, tuples, ranges, etc.

As a trivial example, you can use list comprehension to copy a list:

```
1 a = [1, 4, 6, 2]
2 print(a)
3 d = [x for x in a]
4 print(d)
```

For a slightly more useful example, imagine we want to create a new list whose members are all double the values of the members of list `a`. We could do this using a for loop:

```
1 a = [1, 4, 6, 2]
2 c = [] # Initialise e as an empty list
```



```
3 for i in range(len(a)):
4     c.append(2 * a[i])      # Double the i-th member of a and append to the list
```

Using list comprehension:

```
1 a = [1, 4, 6, 2]
2 c = [2*x for x in a]      # Iterate through a and create new list of 2* the values
```

The second case is more concise and has a small performance advantage in terms of execution time (although you would need a much much longer list to notice this). The most significant advantage is that the code is more readable and easier to see, on a single line of code, the steps taken to create the new list.

`enumerate()` and `zip()` functions can also be used in list comprehension. For example, to perform the calculation from the previous section:

```
1 a = [1, 4, 6, 2]
2 b = [3, 2, 4, 8]
3
4 c = [sum(z) for z in zip(a,b)]  # for each tuple containing corresponding members of
5                                # a and b, add together and put the value into list, c
```

Exercise: Taking the same data as the last exercise:

```
V1 = [1.96 4.02 6.09 8.12 9.91]
V2 = [2.03 3.87 5.79 7.98 9.69]
V3 = [1.98 3.78 6.11 7.93 10.01]
```

Rewrite your script to calculate V_{av} but using list comprehension.

2.6.1 Conditional List Comprehension

List comprehension can be used to filter an existing sequence, only adding items to the new list if a condition is met. The modified syntax is:

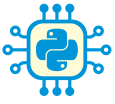
```
new_list = [new_item for member in existing_sequence if condition]
```

For example, to find the integer values of x where $-x^2 + 10x - 12 > 0$:

```
1 sol = [x for x in range(10) if -x*x+10*x-12 > 0]
2 print(sol)
```

This should give the answer:

```
[2, 3, 4, 5, 6, 7, 8]
```



For another example using strings rather than integers, starting with a tuple of strings, to create a new list with just the strings of length four characters:

```
1 g1 = ('peter', 'steve', 'mike', 'tony', 'phil')
2 print(g1)
3 g2 = [m for m in g1 if len(m)==4]      # Only include members of length, 4.
4 print(g2)
```

The resulting output should be:

```
('peter', 'steve', 'mike', 'tony', 'phil')
['mike', 'tony', 'phil']
```

i.e. 'peter' and 'steve' are skipped from the new list because they are both five characters long, not four.



Video Guide: A more complex conditional list comprehension example.
https://mmutube.mmu.ac.uk/id/1_amzjhig9

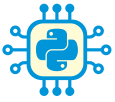
Exercise: Starting from [this list of the largest moons of Uranus](#):

```
moons = ['Miranda', 'Ariel', 'Umbriel', 'Titania', 'Oberon']
```

Use list comprehension to produce the following lists:

1. A list of the second letter of each moon.
2. A list of the length of each moon name.
3. A list of strings containing the first four letters of each moon whose name is longer than six letters.

Outputs should be: 1: ['i', 'r', 'm', 'i', 'b'], 2: [7, 5, 7, 7, 6], 3: ['Mira', 'Umbr', 'Tita']



3 File Input and Output

So far, everything that your scripts have outputted has been to the Shell console and everything that has been inputted has been typed at the keyboard. A very frequent requirement is to take input from a file and/or to output results by writing to a file. There are many formats that files can take, we will look at the most common.

3.1 Writing Text to a File

Writing text (or any other kind of data) to a file generally involve three steps:

1. Open the file
2. Write to the file
3. Close the file

A simple example of this process is:

```
1 outputFile = open('testfile.txt','w')      # open 'testfile.txt' for writing ('w')
2 outputFile.write('Testing file writing.')
3 outputFile.close()
```

This script creates and opens the file, *testfile.txt*, writes a single line of text to it, and then closes the file. If you run this script, you should see the new file appear in your current working directory. In Thonny, from the *View* menu, make sure *Files* is ticked in order to see the current directory and its contents.

A preferred form of the same code using the `with` statement is:

```
1 with open('testfile.txt','w') as outputFile:
2     outputFile.write('Testing file writing.')
```

Note that the `with` statement opens a block of code. Once this block is finished, the file is automatically closed. The advantage of this form is that the file is guaranteed to be closed properly, even if an error causes an exception during the `with` block of code.

You can write multiple lines to a file using more than one call to the write method:

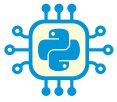
```
1 with open('testfile.txt','w') as outputFile:
2     outputFile.write('File test line 1.\n')
3     outputFile.write('File test line 2.\n')
4     outputFile.write('File test line 3.\n')
```

Note that, unlike the `print()` function, the `write()` method does not start a new line after each call. This is why there is a `'\n'` at the end of each line - code for 'newline'. There are several, so-called, *escape characters* that either cannot be typed or would cause errors in the code that are represented by the backslash character followed by one or more letters. For the full list, see: https://docs.python.org/3/reference/lexical_analysis.html#id11.

Loops are often used to create large text files. For example, using a for loop and formatted string:

```
1 with open('testfile.txt','w') as outputFile:      # Create and open file
2     for i in range(1, 11):                        # Repeat from i = 1 to i = 10
3         outputFile.write(f'File test line {i}.\n') # Write a line to the file
```

[Source Code](#)



Exercise: In the first session, you wrote a script that lists all the powers of 3 (i.e. 1, 3, 9, 27, 81, etc.) less than one million (refer to last weeks notes for details).

Modify this script so that it creates a file called 'powersOfThree.txt' and writes the numbers to it instead of printing to the console. Open the resulting file to check it is correct.

3.2 Reading Text from a File

Reading text from a file follows a similar open-interact-close procedure to writing. When opening, the second argument changes from 'w' to 'r' denoting that the file is opened for reading rather than writing. For example:

```
1 with open('testfile.txt','r') as inputFile:    # 'r' = open for reading
2     text = inputFile.read()                    # Read the entire file...
3     print(text)                               # ...and print to the console
```

[Source Code](#)

Assuming you have already executed the previous examples, you should already have the file 'testfile.txt'. This script will read the entire contents of the file into the variable, `text`, and then print it to the console.

3.2.1 Reading Line by Line

With big files, it is often easier to process the input data one line at a time rather than reading in everything at once. There are several ways to do this but the simplest is by the use of a for loop:

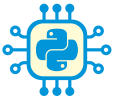
```
1 with open('testfile.txt','r') as inputFile:
2     for line in inputFile:                # Iterate through each line of the file
3         print(line)                       # Print the line
```

Note that the output lines now seem to be double-spaced. This is because each line includes the newline ('\n') character and the print statement adds a newline itself too. To read just the text without the newline at the end, the `rstrip()` string method can be used; it removes all whitespace and non-printing characters from the end of a string:

```
1 with open('testfile.txt','r') as inputFile:
2     for line in inputFile:                # Iterate through each line of the file
3         line = line.rstrip()              # Remove non-printing chars from end of string
4         print(line)                       # and print the result
```

[Source Code](#)

Exercise: From the last exercise, you should already have a file called 'powersOfThree.txt'. Write a script that inputs this file, line by line and prints them to the console.



3.3 Reading and Writing Structured Data

Unless your dataset is just a list of words, a text file may not be the most appropriate format. For example, the bill of materials example in section 1.5 has a list, `bom`, containing dictionaries with strings, floats and integers as members. There are many possible formats for saving data like this but two of the most useful and commonly used are Comma Separated Values (.csv) and JavaScript Object Notation (.json).

3.3.1 Comma Separated Values (.csv)

A .csv file is a text file that has been formatted so that each line represents a row on a spreadsheet; the cells on each row are separated by commas. Cells containing strings can, optionally, have their values enclosed in quotation marks to avoid ambiguities.

For example, taking the bill of materials from section 1.5:

Component	Value	Quantity
Resistor	1000	2
Capacitor	1e-6	1

Could be represented as a .csv file as:

```
Component,Value,Quantity
Resistor,1000,2
Capacitor,1.00E-06,1
```

It is possible to read and write .csv files using built-in string methods to format the output and parse the input. For example:

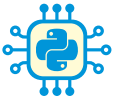
```
1 items = (('Resistor', 1000, 2), ('Capacitor', 1e-6, 1))
2 with open('bom.csv', 'w') as outputFile:
3     outputFile.write('Component,Value,Quantity\n')
4     outputFile.write(f'{items[0][0]},{items[0][1]},{items[0][2]}\n')
5     outputFile.write(f'{items[1][0]},{items[1][1]},{items[1][2]}\n')
```

Try this script out and take a look at the resulting file: `bom.csv`. You should be able to read and edit this using Excel or any other spreadsheet application.

An easier and more reliable way to work with csv files is to make use of the built-in csv library module. A *module* is a library that provides additional functionality to the basic Python language. We will look at more examples of these next time. To use the csv module to create the same file as above:

```
1 import csv # Import the module - don't miss this out!
2
3 items = (('Resistor', 1000, 2), ('Capacitor', 1e-6, 1))
4 with open('bom.csv', 'w', newline='') as outputFile:
5     writer = csv.writer(outputFile)
6     writer.writerow(('Component', 'Value', 'Quantity')) # Write column headings
7     writer.writerows(items) # Write bom data
```

[Source Code](#)



This may not look much easier than the first method, however, reading the file back again is made much easier using the csv module:

[Source Code](#)

```
1 import csv
2
3 items = [] # Empty list to begin with
4 with open('bom.csv','r') as inputFile:
5     reader = csv.reader(inputFile)
6     columns = next(reader)
7     for row in reader:
8         items.append(row)
9
10 print(columns) # Shows the column headings
11 print(items) # The original bom data
```

As we will see in a moment, there are limitations to the types of data that can be stored as csv files. A significant advantage of them is that they can be opened, viewed and edited using any spreadsheet application such as MicroSoft Excel.

3.3.2 JavaScript Object Notation (.json)

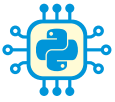
JavaScript Object Notation is a more flexible data format than csv and is widely used for internet transfer and storage of datasets. Unlike .csv, which requires data to be organised in a grid format, JSON data can be organised in arbitrarily dimensioned lists and/or dictionaries with arbitrary sets of keys. For example, a more varied bill of materials might look like this:

```
[
    {"Item":"Resistor", "Value":1000, "Quantity":2},
    {"Item":"Sheet", "Material":"Steel", "Dimensions":{"Length":300, "Width":100}},
    {"Item":"Adhesive", "Type":"PVA", "Volume":20}
]
```

This would be difficult to represent in .csv format due to the variety of keys used in each item's dictionary. The JSON form of this data, however, is exactly the same as the Python notation. To convert a dictionary or list to a JSON formatted string, the `json` module can be used:

[Source Code](#)

```
1 import json
2
3 # Create the dataset to be output
4 bom = [{"Item":"Resistor", "Value":1000, "Quantity":2},
5         {"Item":"Sheet", "Material":"Steel", "Dimensions":{"Length":300, "Width":100}},
6         {"Item":"Adhesive", "Type":"PVA", "Volume":20}]
7
8 with open('bom.json','w') as outputFile: # Create and open the output file
9     outputFile.write(json.dumps(bom)) # Convert to json format and write to file
```

Note that the file creation code is just two lines at the end of the script. To read a json file is similarly easy:

[Source Code](#)

```
1 import json
2
3 with open('bom.json', 'r') as inputFile:      # Open file for reading
4     bom = json.loads(inputFile.read())        # Read entire file and convert from json
5
6 print(bom)                                   # Show the result
```

Exercise: Download the file [moons.json](#) and save it in your current working directory. It contains data on all 28 moons of the planet Uranus. For each moon a dictionary is stored with members: 'Name' - the moon's name; 'Diameter' - diameter in km; 'Orbital Distance' - distance from Uranus in km; and 'Length of Orbit' - orbital period in days.

1. Write a script to read the file, `moons.json`, and store in a variable, `moons`.
2. Add a for loop that iterates through the list, `moons`, printing the name and length of orbit for each moon to the console.
3. Add code to the script to create a second list, `largeMoons`, that only contains the data for moons with a diameter greater than 100 km. Use list comprehension to build this list.
4. Add code to write the data in `largeMoons` to a json formatted file: `largeMoons.json`.

The resulting file, `largeMoons.json`, can be difficult to interpret but tools such as online viewers like <https://jsonformatter.org/> or VS Code extensions like [JSON Diagram Viewer](#) can help. The JSON file should contain the data for the nine largest moons: Juliet, Portia, Puck, Miranda, Ariel, Umbriel, Titania, Oberon and Sycorax.

Task: Write a script to read the same file, `moons.json`, and prints the name of each moon that has an orbital distance less than 65000 km.

Enter or upload your script using the 'Closest Moons Task' in this week's Moodle section for testing and assessment.